

软件过程改进方案设计文档

一、报告说明

本文档以前期完成的“辞典编撰辅助平台开发”软件过程定义和《AI技术调研与场景分析》为基础，围绕作业要求中的“软件过程改进方案设计”展开。原软件过程将项目划分为领域需求与数据准备、核心算法与引擎研发、系统架构与增量开发、人机协同与闭环验证、规范定稿与部署发布五个阶段。该过程能够覆盖辞典编撰辅助平台从需求理解到系统交付的主要活动，但在面对生成式AI、Graph-RAG、多智能体和专家协同等新型工程特征时，还需要进一步明确AI工具如何嵌入过程、AI输出物如何验证、AI风险如何控制。中国信通院的AI4SE报告与智能化软件开发实践指南均指出，AI正在从单点工具走向软件研发全生命周期过程能力，因此过程层面的重构比单纯工具试用更重要[1][2]。

本项目的目标系统是“基于训诂学的汉字释义知识图谱构建与辞典编撰辅助平台”。系统面向辞典编撰专家和古汉语研究者，通过Neo4j、MySQL、React、FastAPI、Graph-RAG、大模型和多智能体等技术，支撑“审、比、勘、改”的训诂思维链。由于项目同时具有领域知识复杂、证据链要求高、小团队工程实现任务重、AI输出需要专家复核等特点，过程改进不能简单理解为“使用AI写代码”，而应形成覆盖需求、设计、编码、测试和运维的AI增强型软件过程。

根据步骤一的分析，本轮过程改进优先选择三个核心环节：AI辅助需求分析、AI驱动的代码生成与审查、AI辅助测试用例与质量分析。在此基础上，适度扩展到系统设计、模型质量监控和持续改进环节，形成一套具备可落地性、可验证性和可度量性的改进方案。该选择也与近年研究中“AI工具需要配套组织过程、质量门禁和工程治理”的观点一致[4][6]。

二、改进目标与原则

本次软件过程改进的总体目标是：在保留原有生命周期过程主干的前提下，引入AI工具、AI角色和AI质量门禁，使团队能够更高效地完成跨学科需求理解、系统实现和质量验证，同时保证辞典编撰场景所要求的证据可追溯、专家可复核和结果可解释。

具体目标包括以下五点：

- 提升需求理解效率。** 利用大语言模型和RAG技术辅助研读课程材料、专家访谈、辞书体例和训诂学文献，形成需求条目、术语表、验收标准和追溯关系，减少人工整理成本。
- 提升工程实现效率。** 在React前端、FastAPI后端、数据库访问、接口封装、提示词模板和测试脚本等环节引入AI辅助编码与代码审查，降低小团队重复性开发负担。
- 提升测试设计覆盖度。** 使用AI根据需求规约、接口说明、典型字头数据和异常场景自动生成测试用例，并辅助发现遗漏的边界条件、证据缺失路径和专家验收点。
- 强化AI输出质量控制。** 对AI生成的需求、代码、测试用例、Agent结论和运维分析建立质量标准，明确人工复核、自动化测试、专家确认和追溯记录机制。
- 形成持续改进闭环。** 将专家反馈、测试结果、模型幻觉记录、代码缺陷和运行日志纳入度量体系，支撑后续迭代优化。

改进过程遵循以下原则：

原则	具体含义
人机协同原则	AI负责生成草稿、建议和辅助分析，人负责关键判断、确认和最终决策
证据约束原则	涉及辞书释义、书证、引申关系和修订建议的AI输出必须绑定来源证据
小步迭代原则	不一次性改造全部流程，先在需求、编码、测试三个高价值环节落地
可度量原则	为AI辅助活动设置效率、质量、采纳率、缺陷率等指标，避免只凭主观感受判断效果
可追溯原则	保留AI提示词、输入材料、输出结果、人工修改意见和专家反馈记录
安全合规原则	对辞书资料、专家意见、模型接口密钥和项目数据进行权限控制与脱敏处理

三、改进后总体过程模型

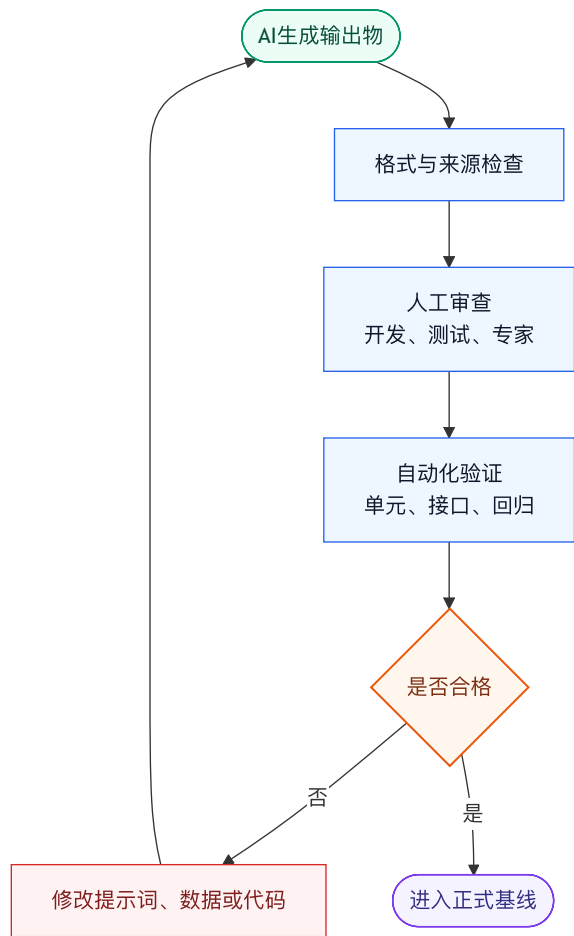
改进后的软件过程仍以原五阶段生命周期为主线，但在每个阶段加入AI参与节点和质量门禁。其核心变化是：需求阶段由AI辅助完成资料理解和需求初稿生成；设计阶段由AI辅助生成架构方案、接口草案和流程图；编码阶段由AI辅助代码生成与审查；测试阶段由AI辅助测试用例、测试数据和质量分析；运维阶段由AI辅助日志摘要、异常分析和模型质量监控。所有AI输出均不能直接作为最终成果，必须经过相应的人工审查、自动化测试或专家确认。

1. AI增强软件过程总览图



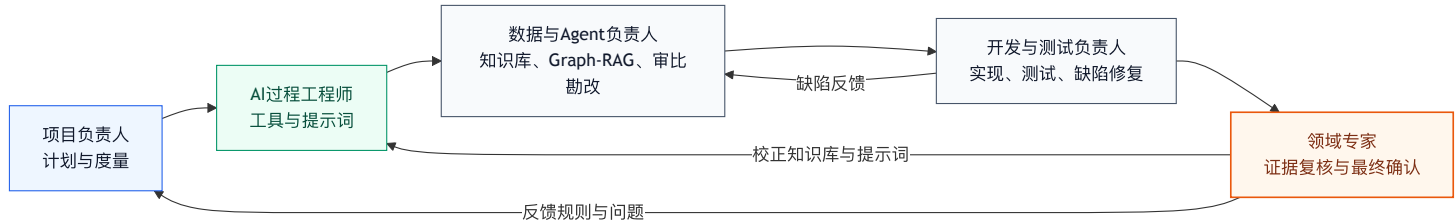
从流程图可以看出，改进后的过程不是线性结束的瀑布模型，而是一个带有反馈闭环的迭代过程。AI主要参与需求、设计、编码、测试和监控环节，但每次进入交付或部署前都必须经过人工评审、自动化测试或专家确认。

2. AI输出物质量门禁流程图



该流程图强调AI输出物的正式化条件：先检查格式与来源，再由人工审查，最后通过自动化验证。只有同时满足这些条件的内容，才能进入需求基线、代码主干、测试集或专家修订记录。

3. 小团队角色协作流程图



该流程图体现小团队“一人多职、模块负责、专家闭环”的协作方式。项目负责人控制迭代节奏，AI过程工程师维护AI使用规范，数据、Agent、开发和测试人员完成工程落地，领域专家负责最终学术确认。

四、关键环节过程重构

1. AI辅助需求分析过程

原过程中的“领域需求与数据准备过程”主要依赖人工阅读项目背景、课程要求和专业材料，再将其转化为软件需求。该方式适合规模较小、需求较清晰的普通系统，但本项目具有明显的跨学科特征：训诂学概念、辞书体例、多源书证、义项关系、专家修订习惯都需要被理解和结构化。若完全依赖人工整理，容易出现术语不统一、需求遗漏、隐性规则难表达、验收标准不清晰等问题。

改进后，在需求阶段引入“大模型+私有知识库/RAG”的辅助流程。团队先将课程作业要求、第一次软件过程定义文档、AI技术调研文档、辞书编撰背景、专家访谈纪要和典型字头样例整理为需求知识库。AI工具根据这些材料生成需求摘要、领域术语表、功能需求列表、非功能需求列表、用户故事和初步验收标准。随后由项目成员和训诂学专家进行逐条审核，确认哪些内容可以进入正式需求基线。

该过程的输入、活动和输出如下：

项目	内容
输入	作业要求、第一次软件过程定义、步骤一调研报告、专家访谈纪要、辞书体例、典型字头样例
AI活动	文档摘要、术语抽取、需求条目生成、需求冲突提示、验收标准草拟
人工活动	术语校正、需求优先级排序、隐性规则补充、需求基线确认
输出	需求规格说明、术语表、需求追溯矩阵、专家确认记录、验收标准清单
质量标准	每条关键需求应能追溯到文档出处或专家意见；领域术语应经过人工校正；AI生成需求不得直接进入基线

例如，对于“审、比、勘、改”训诂思维链，AI可以先根据背景材料生成四个阶段的初步需求：审阶段完成单源义素拆解，比阶段完成多源义项对齐，勘阶段生成待勘问题和证据链，改阶段输出修订建议与修改日志。但这些需求是否符合真实辞典编撰流程，必须由领域专家确认。这样既能利用AI提高整理效率，又能避免模型误解专业概念。

2. AI辅助系统设计过程

在原过程中，核心算法与引擎研发主要围绕知识图谱、Graph-RAG和多智能体进行设计。改进后，AI不直接替代架构设计决策，而是作为设计助手参与方案生成、风险提示和文档化表达。团队可以使用AI根据需求条目生成模块划分建议、接口草案、Mermaid流程图、数据库字段候选项和Agent职责说明，再由架构负责人进行筛选。

本项目中AI辅助设计的重点包括四类：第一，辅助梳理“前端页面-后端接口-图数据库-模型服务”的调用链路；第二，辅助生成Graph-RAG的检索流程、证据组装流程和异常处理流程；第三，辅助定义保守派Agent、革新派Agent、裁判Agent的输入、输出、工具权限和日志字段；第四，辅助形成接口文档、数据字典和流程图。

该环节的质量门禁是“设计可解释、接口可实现、数据可追溯”。AI生成的架构建议如果无法说明数据来源、调用关系和异常处理方式，不应进入设计基线。对于涉及学术判断的Agent设计，还需要明确其输出必须包含证据、推理过程、置信度和待专家确认标记，避免模型用确定口吻输出未经证实的结论。

3. AI驱动的代码生成与审查过程

原过程中的“系统架构与增量开发过程”已经采用小团队敏捷迭代方式。改进后，在编码实现阶段引入GitHub Copilot、通义灵码、CodeGeeX或Qwen Coder类代码助手，用于生成低风险、结构明确、可测试的工程代码。可优先使用AI生成React组件、FastAPI路由、请求参数校验、Neo4j/MySQL查询封装、接口文档注释、单元测试样例和错误处理模板。GitHub Copilot等工具已经覆盖代码生成、解释、测试和开发问答等任务，但其输出仍需要开发者审查和测试验证[7][8]。

AI辅助编码的流程如下：

- 开发者根据需求和接口设计编写明确任务说明，包括输入、输出、边界条件和错误处理要求。
- AI工具生成代码初稿，开发者对关键逻辑、依赖调用、异常处理和安全问题进行人工审查。
- 对AI生成代码运行格式化、静态检查、单元测试和接口测试。
- 代码提交前必须说明哪些部分由AI辅助生成，记录提示词要点和人工修改点。
- 通过代码评审后才能合并到主分支。

在本项目中，AI适合辅助实现以下模块：

模块	AI可辅助内容	人工重点审查
React前端	字头检索页、义项对齐表、书证展示卡片、专家反馈表单	状态管理、交互逻辑、中文显示、异常提示

模块	AI可辅助内容	人工重点审查
FastAPI后端	查询接口、参数校验、响应模型、错误处理	权限控制、输入校验、接口一致性、性能
数据访问	Neo4j查询模板、MySQL读写封装、数据转换函数	查询正确性、注入风险、索引使用、事务边界
Agent编排	角色配置样例、调用链模板、输出JSON结构	提示词边界、工具权限、证据引用、日志记录
测试脚本	Pytest接口测试、Playwright页面测试、模拟数据	测试断言、边界覆盖、可复现性

该环节的质量标准是：AI生成代码不能绕过人工Review；关键业务逻辑必须有测试覆盖；涉及数据库、模型调用、文件读写和密钥配置的代码必须进行安全审查；生成代码若无法解释其实现意图，不应直接纳入项目。

4. AI辅助测试用例与质量分析过程

原过程中的“人机协同与闭环验证过程”强调自动化测试和专家UAT。改进后，将AI引入测试设计、测试数据生成、缺陷分析和专家验收清单生成。由于本项目测试对象既包括普通软件功能，也包括AI推理链质量，因此测试过程需要同时覆盖接口正确性、页面可用性、证据链完整性、Agent输出格式和专家可复核性。智能测试能力标准强调整测试过程中的能力分级、工具支撑和质量评估，这为本项目建立AI辅助测试门禁提供了参考[3]。

AI辅助测试的典型流程为：

- 将需求规格、接口说明、典型字头数据和异常场景输入AI工具。
- AI生成测试用例草案，覆盖正常路径、边界条件、异常输入、证据缺失、检索无结果、模型输出格式错误等情况。
- 测试人员筛选、修改和补充测试用例，形成正式测试集。
- 对接口测试、页面测试和Agent输出测试进行自动化执行。
- AI辅助分析失败用例、缺陷聚类 and 风险模块，测试人员确认根因并提交缺陷。
- 专家根据验收清单复核“审比勘改”结果，并给出采纳或退回意见。

本项目可建立如下测试资产：

测试资产	内容示例	AI作用
接口测试用例	输入字头后返回义项、书证、来源、置信度等字段	根据接口文档生成正常与异常用例
Graph-RAG测试集	检索结果缺少权威书证时不得输出确定性修订建议	设计证据缺失、证据冲突、重复证据场景
Agent输出测试	保守派、革新派、裁判Agent均需输出立场、证据、结论和置信度	生成格式校验规则和反例
UI验收清单	专家可查看义项对齐、书证来源、辩论过程和修改日志	根据用户故事生成验收项
回归测试集	典型字头、典型书证、典型义项对齐案例	根据历史缺陷补充测试点

测试阶段的AI输出同样需要质量门禁。AI生成的测试用例应由测试人员抽查和修改；测试断言应尽量基于明确字段、状态码、证据数量和格式规则；对于训诂学判断类结果，应交由专家确认，不应只用模型自评作为通过依据。

5. AI辅助运维监控与持续改进过程

原过程的最后阶段是规范定稿与部署发布。改进后，在部署运行阶段加入AI辅助运维监控，使系统不仅关注接口是否可用，也关注AI链路是否可靠。系统运行后应持续记录Graph-RAG检索命中率、模型响应延迟、Agent输出格式合格率、专家采纳率、幻觉或误引书证次数、接口错误率和用户反馈。

AI可以辅助完成日志摘要、异常聚类、根因分析和改进建议生成。例如，当某类字头经常出现检索证据不足时，AI可以提示团队检查知识库覆盖范围或检索策略；当某个Agent频繁输出格式不合格时，AI可以帮助分析提示词约束是否不足；当专家采纳率下降时，团队需要回溯需求、数据、提示词和测试集，判断问题来源。

五、新增角色与职责定义

为支撑AI增强型软件过程，原有小团队分工需要增加若干AI相关职责。考虑团队规模约四人，新增角色不一定对应专职人员，可以由成员兼任，但职责边界需要明确。

角色	主要职责	关键交付物
项目负责人/过程负责人	维护整体软件过程、迭代计划、质量门禁和度量指标	迭代计划、过程度量表、风险记录
领域专家/需求确认人	审核训诂学需求、辞书体例、义项判断和最终修订建议	专家确认记录、需求变更意见、UAT结论

角色	主要职责	关键交付物
AI过程工程师	设计AI嵌入节点、维护提示词模板、评估AI辅助效果	AI使用规范、提示词模板、AI输出评估记录
知识库与数据工程负责人	管理辞书语料、书证材料、术语表、向量库和图数据库	数据字典、清洗规则、知识库更新记录
Agent设计负责人	设计保守派、革新派、裁判Agent的职责、权限和输出格式	Agent配置、提示词版本、辩论日志样例
开发工程师	使用AI辅助完成前后端、接口、数据库和模型服务开发	源代码、接口文档、代码审查记录
AI测试分析师	使用AI生成测试用例、分析缺陷、维护专家验收清单	测试用例集、测试报告、缺陷分析报告
配置与安全负责人	管理代码版本、模型密钥、数据权限、日志脱敏和依赖版本	配置清单、安全检查记录、发布记录

其中，AI过程工程师和AI测试分析师是本次改进中最重要的新增职责。AI过程工程师负责保证AI工具使用有规范、有记录、有质量门禁；AI测试分析师负责把AI生成能力转化为可执行的测试资产，而不是停留在“让模型评价模型”的不可靠状态。

六、AI工具链选型与集成方案

结合本项目的技术路线和作业要求，建议采用“通用AI工具+私有知识库+现有工程工具”的组合式工具链。工具选型不追求一次性全栈自研，而是优先选择成熟、可集成、可替换的工具，在关键环节保留人工复核。

改进环节	推荐工具或技术框架	核心功能	与现有过程的集成方式	预期效率提升指标
需求分析	Qwen/ChatGPT类大模型、RAG知识库、Markdown模板	文档摘要、术语抽取、需求条目生成、验收标准草拟	接入课程文档、专家访谈和辞书体例，生成需求草稿后人工评审	需求初稿整理时间减少约30%-50%，需求追溯覆盖率达到90%以上
系统设计	Mermaid、PlantUML、LLM架构助手、LlamaIndex/LangChain	流程图生成、模块划分、接口草案、Agent编排建议	根据需求基线生成设计草案，由架构负责人筛选和修订	设计文档初稿产出时间减少约25%-40%
编码实现	GitHub Copilot、通义灵码、CodeGeeX、Qwen Coder	代码生成、补全、解释、重构和代码审查	嵌入开发IDE，用于React、FastAPI、数据库访问和测试脚本初稿	常规工程代码实现时间减少约20%-35%，AI代码采纳率控制在可审查范围内
代码质量	SonarQube、ESLint、Ruff、MyPy、GitHub Pull Request Review	静态分析、规范检查、潜在缺陷提示	与提交和合并流程绑定，AI生成代码必须通过检查	主干分支阻断明显低级缺陷，减少空值、格式和重复逻辑问题
测试验证	LLM+Pytest、Postman、Playwright、Testin XAgent类智能测试工具	测试用例生成、接口测试、UI测试、测试结果分析	根据需求和接口文档生成测试用例，经人工确认后执行	测试用例设计效率提升30%以上，关键需求测试覆盖率达到85%以上
运维监控	日志RAG、AIOps异常检测、Prometheus/Grafana	日志摘要、异常聚类、响应延迟与模型质量监控	采集接口日志、模型调用日志、专家反馈和采纳率数据	常见异常定位时间减少约30%，形成可持续改进数据

上述指标属于本项目的预期管理目标，并不是无条件保证值。实际效果需要在第三步“AI技术应用验证报告”中选择一个核心场景进行最小可行验证，再根据验证数据进行修正。

七、AI输出物质量标准与验证机制

AI引入后，软件过程中的交付物类型会发生变化。除了传统需求文档、设计文档、代码、测试报告之外，还会新增提示词模板、AI生成记录、知识库索引、Agent配置、模型输出日志和AI辅助测试集等内容。为了保证过程可控，需要为不同AI输出物设置明确的质量标准。

AI输出物	质量标准	验证机制
需求条目草稿	表述清晰、来源可追溯、与项目目标一致、无明显冲突	需求评审、专家确认、追溯矩阵检查
领域术语表	术语名称统一、含义准确、与辞书体例一致	领域专家审核、版本记录
架构与流程图草案	模块边界清晰、数据流完整、异常路径明确	架构评审、接口走查、Mermaid语法检查

AI输出物	质量标准	验证机制
AI生成代码	可读、可测试、符合项目规范、无明显安全风险	人工Review、静态扫描、单元测试、集成测试
测试用例草案	覆盖正常、边界、异常和专家验收场景，断言明确	测试人员审核、需求覆盖率检查、实际执行
Agent输出	必须包含立场、证据、推理依据、结论、置信度和待确认标记	格式校验、证据溯源、专家复核
运维分析报告	基于真实日志和指标，不夸大因果关系，能定位改进方向	指标核对、人工确认、缺陷复盘

特别需要强调的是，AI不能成为辞典编撰结论的最终裁判。对于“某个义项是否应合并”“某条书证是否支持修订”“某种引申关系是否成立”等问题，AI只能提供证据组织和分析建议，最终结论必须经过专家确认。

八、风险评估与应对策略

AI技术引入能够提升效率，但也会带来新的过程风险。本项目涉及古籍语料、专家知识、大模型推理和软件工程交付，风险控制尤其重要。

风险类型	风险表现	影响	应对策略
数据安全风险	专家访谈、辞书材料、模型密钥或项目数据被上传到不受控平台	造成数据泄露或版权争议	优先使用私有化部署或受控API；敏感材料脱敏；密钥不进入提示词和代码仓库
模型幻觉风险	AI编造书证、误解古汉语术语、输出未经证实的修订建议	影响学术严谨性和专家信任	引入RAG证据约束；要求输出引用来源；专家复核；记录幻觉案例用于改进
模型偏见与知识局限	通用模型倾向现代汉语解释，忽视训诂学语境	导致义项判断偏差	构建领域知识库；使用典型反例测试；由专家校正提示词和术语表
过度依赖风险	团队直接采纳AI生成代码、需求或测试用例	产生隐蔽缺陷和需求偏差	明确AI输出为草稿；设置Review和测试门禁；关键交付物必须人工签字确认
工具锁定风险	过程严重依赖某一AI工具或接口	后续迁移困难，成本不可控	采用可替换接口；保留Markdown、JSON、Mermaid等开放格式；记录工具版本
提示词漂移风险	提示词修改未记录，导致Agent行为前后不一致	难以复现和定位问题	对提示词、Agent配置和知识库版本进行配置管理
质量评估失真风险	只用AI自评AI输出质量	测试结论不可靠	结合自动化测试、人工审查、专家验收和真实运行指标
性能与成本风险	模型调用延迟高、费用增加、并发能力不足	影响系统可用性和项目预算	设置缓存、限流和降级策略；监控调用成本；重要链路进行性能测试

九、落地计划与过程度量指标

为了保证改进方案可落地，建议采用三轮迭代逐步实施。

第一轮迭代聚焦需求阶段。目标是建立需求知识库、需求生成模板和专家确认流程。完成后应产出需求条目、术语表、验收标准和追溯矩阵，并记录AI生成内容的人工修改率。

第二轮迭代聚焦编码阶段。目标是在一个小功能闭环中使用AI辅助完成前端页面、后端接口、数据库查询和基础测试脚本，例如“输入字头-查询义项-展示书证-提交专家反馈”的最小流程。完成后统计开发耗时、AI代码采纳率、人工修改点和缺陷数量。

第三轮迭代聚焦测试阶段。目标是使用AI根据需求和接口文档生成测试用例，再由测试人员筛选执行，形成对比数据。该轮结果可直接服务于后续第三步“AI技术应用验证报告”。

建议设置如下过程度量指标：

指标类别	指标名称	目标值或观察方式
需求效率	需求初稿生成耗时	相比纯人工减少30%以上
需求质量	需求追溯覆盖率	关键需求90%以上可追溯到资料或专家意见
编码效率	常规代码实现耗时	相比纯人工减少20%-35%

指标类别	指标名称	目标值或观察方式
编码质量	AI生成代码缺陷率	不高于人工代码，且必须通过Review与测试
测试效率	测试用例设计耗时	相比纯人工减少30%以上
测试覆盖	关键需求测试覆盖率	达到85%以上
模型质量	Agent输出格式合格率	达到90%以上
学术可靠性	修订建议专家采纳率	按迭代持续跟踪，不以单轮绝对值判断
运维质量	平均异常定位时间	通过日志摘要和异常聚类持续下降

十、与原软件过程定义的对应关系

本次改进并不是推翻原有过程，而是在原过程基础上进行AI增强。对应关系如下：

原软件过程阶段	改进后新增AI能力	改进效果
领域需求与数据准备	AI辅助文档研读、术语抽取、需求条目生成、追溯矩阵建立	降低跨学科理解成本，提高需求完整性
核心算法与引擎研发	AI辅助Graph-RAG流程设计、Agent角色设计、提示词模板生成	提高原型设计效率，强化输出格式约束
系统架构与增量开发	AI辅助前后端代码、接口封装、数据库访问和代码审查	提高小团队开发效率，减少重复劳动
人机协同与闭环验证	AI辅助测试用例、异常场景、专家验收清单和缺陷分析	提高测试覆盖度和验证可复核性
规范定稿与部署发布	AI辅助日志分析、异常检测、模型质量监控和持续改进	将运行数据反哺过程优化

经过改进后，项目的软件过程从“传统生命周期+领域剪裁”进一步发展为“AI增强生命周期+专家闭环治理”。该过程既保留了ISO/IEC 12207所强调的需求、设计、实现、验证、交付和维护主线，也吸收了AI4SE对自动化、智能化和持续度量的要求，更符合辞典编撰辅助平台这种跨学科智能系统的实际需要。

十一、总结

本文提出的软件过程改进方案，以步骤一确定的AI应用场景匹配结果为基础，围绕需求分析、编码实现和测试验证三个核心环节进行重点设计，并扩展到系统设计和运维监控环节。改进后的过程通过新增AI过程工程师、AI测试分析师、知识库与数据工程负责人等职责，将AI工具嵌入需求生成、架构设计、代码实现、测试分析和持续运维中，同时设置人工审查、专家复核、自动化测试和度量指标作为质量门禁。

对于辞典编撰辅助平台而言，AI的核心价值不是替代专家，也不是完全自动生成系统，而是帮助团队更快地整理专业知识、更高效地实现工程功能、更系统地设计测试和更持续地改进过程。只有在证据约束、专家确认和过程度量的共同作用下，AI技术才能真正成为软件过程改进的可靠工具。

参考文献

[1] 中国信息通信研究院人工智能研究所. AI4SE行业现状调查报告（2026年）[R/OL]. 北京: 中国信息通信研究院, 2026[2026-05-30]. <https://www.caict.ac.cn/kxyj/qwfb/ztbg/202604/P020260402599647343221.pdf>.

[2] 中国信息通信研究院, 华为云. 智能化软件开发落地实践指南（2024）[R/OL]. 北京: 中国信息通信研究院, 2024[2026-05-30]. <https://blockchain.caict.ac.cn/kxyj/qwfb/ztbg/202409/P020240919585073158926.pdf>.

[3] 中国人工智能产业发展联盟. 智能化软件工程技术和应用要求 第3部分: 智能测试能力[S/OL]. AIIA/PG 0138-2024, 2024[2026-05-30]. https://www.sohu.com/a/843331181_121124361.

[4] Gurgul V, Gubela R, Lessmann S. The State of Generative AI in Software Development: Insights from Literature and a Developer Survey[J/OL]. arXiv:2603.16975, 2026[2026-05-30]. <https://arxiv.org/abs/2603.16975>.

[5] Qiu K, Puccinelli N, Ciniselli M, Di Grazia L. From Today's Code to Tomorrow's Symphony: The AI Transformation of Developer's Routine by 2030[J/OL]. arXiv:2405.12731, 2024[2026-05-30]. <https://arxiv.org/abs/2405.12731>.

[6] Google Cloud DORA. State of AI-Assisted Software Development 2025[R/OL]. 2025[2026-05-30]. <https://dora.dev/research/2025/dora-report/>.

[7] Stack Overflow. 2025 Developer Survey: Artificial Intelligence[EB/OL]. 2025[2026-05-30]. <https://survey.stackoverflow.co/2025/ai>.

[8] GitHub. GitHub Copilot Documentation[EB/OL]. 2026[2026-05-30]. <https://docs.github.com/en/copilot>.

